03

MGU BCA - 3RD SEM

Software Engineering

Module 3 - Design, Testing and Metrics

By Aben Sojan

Exam - 10 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

SF HONOURS EXAM PATTERN - 50 marks, 1.5 hours. Part A: 10 x 2 marks, all compulsory; Q8-Q10 come from this module (design and testing). Part B: 5 x 5 marks, answer any 3 of 5. Part C: 2 x 15 marks, answer any 1 of 2; the model paper essay is system testing approaches - the single most likely 15-marker. White-box techniques 5x, black-box techniques 3x, cohesion-coupling 3x, reliability 3x - all sure marks. UML diagrams have zero old-paper precedent but are core 2024-syllabus topics; attempt at least one question from each course outcome (CO3 = this module).

TOPIC 1

Design Process

1

Design turns requirements into a blueprint the coders can build.

The design process moves from high-level architecture down to detailed component design.

Good design is the foundation; bad design makes coding slow and buggy.

QUESTION BANK

1 What is software design? Describe its objectives. [Asked: Aug 2022, Feb 2025]

REPEATED

Software design is the process of planning how the software will be built before coding starts.

Its first objective is to create a blueprint that developers can follow without confusion.

It aims for correctness: the design must faithfully reflect the requirements.

It aims for modularity: breaking the system into independent, manageable parts.

It aims to make future changes cheap by keeping parts loosely connected.

It aims for understandability, so new team members can grasp the system quickly.

It produces documents like architecture diagrams and interface designs that guide coding and testing.

Example: designing a bus-tracking app means deciding its modules, screens, database tables and how GPS data flows, all before writing code.

In short, software design turns requirements into a clear construction plan.

2 Explain the software design process.

IMPORTANT

Software design is the step where the requirements are converted into a clear plan for building the software.

It answers the question: how will the software be built, before any coding starts.

Step 1: Study the requirements document carefully so that nothing is missed.

Step 2: Create the high-level architecture, dividing the system into major parts.

Step 3: Design the data, deciding the tables, files and data structures to use.

Step 4: Design each component in detail, defining its inputs, outputs and internal logic.

Step 5: Design the user interface, deciding the screens and how the user interacts with them.

Step 6: Review the design against the requirements and correct any mistakes.

Only after the design is reviewed and approved does coding begin.

Example: before coding a college portal, designers first draw how the login, fees and exam modules connect.

A good design process saves time, because fixing a design mistake is much cheaper than fixing code later.

3 What are the inputs and outputs of the design process?

IMPORTANT

The input of the design process is the approved software requirements document.

It also takes constraints as input, such as the hardware limits, budget, deadline and technology to be used.

The output of the design process is a complete set of design documents.

These include architecture diagrams showing the major parts and their connections.

They also include data design documents, such as database tables and their relationships.

Interface design documents describe the screens and the way modules communicate.

Component design documents describe the internal logic of each module in detail.

The design documents act as a blueprint that programmers follow while coding.

Before coding starts, the design output is reviewed by the team for errors and gaps.

Example: for a banking app, the output would include the database design for accounts and the screen design for money transfer.

In short, requirements go in as input, and a buildable design comes out as output.

TOPIC 2

2

Design Quality

Design quality means the design is correct, clear and easy to change.

A quality design: does what is required, is modular, is easy to understand and test.

Poor design leads to bugs, delays and painful maintenance.

QUESTION BANK

4 **Define reliability. Explain the ways of characterising failure occurrences in time. [Asked: Mar 2021, Oct 2021, Aug 2022]**

SURE MARK

Reliability is the probability that software works without failure for a given period in a given environment.

It is measured by observing how failures occur over time, not by guessing.

There are four general ways of characterising failure occurrences in time.

Time of failure records exactly when each failure happens.

Time interval between failures measures the gaps between failures, which grow as the software improves.

Cumulative failures plot the total count over time, showing whether failures are slowing down.

Failures experienced in a time interval show the failure rate in each period, like failures per week.

These measures tell the team whether testing is working and when the software is ready to release.

Example: if a server fails after 10, 40 and 100 days, the growing intervals show reliability is improving as bugs are fixed.

In short, reliability is quantified by studying the timing pattern of failures.

5 What are the characteristics of a good design?

IMPORTANT

A good design is one that is correct, easy to understand, easy to change and easy to test.

Correctness: the design fully covers all the requirements with nothing missing or extra.

Modularity: the system is split into independent modules, so one part can be changed without breaking others.

Low coupling: modules depend on each other as little as possible.

High cohesion: each module does one clear job and does it completely.

Understandability: another developer can read the design and understand it without much help.

Simplicity: the design avoids unnecessary complexity; the simplest solution that works is best.

Testability: the design makes it easy to test each module separately.

Reusability: well-designed modules can be reused in future projects.

Example: a good design for an online shop keeps payment, orders and stock as separate modules.

Good design is the foundation; bad design leads to bugs, delays and painful maintenance.

6 How is design quality reviewed before coding?

IMPORTANT

Design quality is checked before coding through a design review meeting.

In a review, the designers present their design documents to a group of reviewers.

Reviewers check whether every requirement is covered by the design.

They look for missing parts, unclear modules and risky decisions.

They check coupling and cohesion, making sure modules are independent and focused.

They also check that the design follows the project standards and constraints.

Any defect found is noted down and the design is corrected before coding starts.

Finding a defect in design review is far cheaper than finding it in code or after delivery.

Example: reviewers may notice that the login module was never connected to the user database, and fix it early.

In short, design review is a quality gate that stops bad design from reaching the coding stage.

TOPIC 3

3

Design Concepts

Key design concepts guide every good designer.

Abstraction: hide complex details, show only what is needed.

Modularity: split the system into independent modules.

Coupling: modules should depend on each other as little as possible (low coupling).

Cohesion: each module should do one job well (high cohesion).

QUESTION BANK

7 **Explain abstraction, modularity, coupling and cohesion. [Asked: Mar 2021, Oct 2021, Aug 2022]**

SURE MARK

These four concepts are the basic rules that guide every good designer.

Abstraction means hiding complex details and showing only what is needed.

It lets a designer work with a simple view of a module without worrying about its inner code.

Modularity means splitting the system into separate independent modules.

Each module can be developed, tested and changed on its own.

Coupling is the degree of dependence between modules; good design keeps coupling low.

With low coupling, changing one module does not break the others.

Cohesion is how focused a single module is; good design keeps cohesion high.

With high cohesion, each module does one clear job completely.

Example: a login module should only handle login, and should not depend on the report module.

In short, good design means simple views, separate parts, low dependence and focused modules.

8 What is cohesion? Why should it be high? [Asked: Model paper]

REPEATED

Cohesion is the measure of how focused a single module is on one job.

In high cohesion, all the parts of a module work together for one clear purpose.

A highly cohesive module is easy to understand, because it does just one thing.

It is also easy to test and reuse, since it has a single clear responsibility.

In low cohesion, a module does many unrelated jobs mixed together.

Low cohesion modules are confusing, hard to test and hard to change safely.

Cohesion should be high because a focused module is reliable and easy to maintain.

Example: a payment module that only handles payments has high cohesion.

If the same module also prints reports and manages user accounts, it has low cohesion.

That is why designers always aim for high cohesion inside each module.

9 What is coupling? Why should it be low?

IMPORTANT

Coupling is the measure of how much different modules depend on each other.

In high coupling, modules are tightly connected and depend on each other's internal details.

With high coupling, changing one module can break another module, even one that seems unrelated.

High coupling also makes testing hard, because a module cannot be tested alone.

In low coupling, modules interact only through simple, well-defined interfaces.

With low coupling, each module can be changed, tested and reused independently.

Coupling should be low because software always changes, and low coupling keeps changes safe and cheap.

Example: if the billing module directly reads the internals of the stock module, they are highly coupled.

If instead billing only calls a simple getPrice() function, the coupling is low.

That is why designers always aim for low coupling between modules.

10 Write about the term module.

IMPORTANT

A module is a separate, independent unit of software that performs one clear task.

Good modules have high cohesion: everything inside works toward one single purpose.

Good modules have low coupling: they depend as little as possible on other modules.

Modules communicate through well-defined interfaces while hiding their internal details.

Breaking software into modules makes it easier to understand, test and change.

A faulty module can be fixed or replaced without rewriting the whole system.

Modules also enable teamwork, since different people can build different modules in parallel.

Example: in a college app, admission, fees and attendance can be three separate modules.

In short, a module is one well-shaped brick, and good software is built from such bricks.

11 Explain the difference between object-oriented and function-oriented design.

IMPORTANT

Object-oriented design organises software around objects; function-oriented design organises it around functions.

In object-oriented design, data and the operations on that data are bundled together into classes.

In function-oriented design, the system is broken into functions that transform input data into output data.

Object-oriented design uses inheritance and polymorphism to reuse code.

Function-oriented design uses data flow diagrams and structure charts to show processing steps.

Object-oriented design suits systems with complex, long-lived data, like banking systems.

Function-oriented design suits systems that are mainly data processing pipelines, like payroll.

Maintenance differs too: objects localise change, while functions may need changes across many steps.

Example: a game with players and enemies fits objects; a salary-slip generator fits functions.

In short, objects group data with behaviour, functions group the steps that process data.

TOPIC 4

Design Model

4

The design model is the complete set of design documents.

It includes: data design, architectural design, interface design and component design.

Each level adds more detail, from overall structure to code-level logic.

QUESTION BANK

12 **Explain the design model and its levels.**

IMPORTANT

The design model is the complete set of design documents that describe the software.

It is created in levels, each level adding more detail than the previous one.

Data design: decides the data structures, database tables, fields and relationships.

Architectural design: decides the overall structure, the major parts and how they connect.

Interface design: decides the user screens and the way different parts communicate with each other.

Component design: decides the internal logic of each part, close to the actual code.

The levels move from the big picture down to the fine details.

Each level is reviewed before moving to the next, so mistakes are caught early.

Example: for a hospital system, architecture first splits it into patient, doctor and billing parts, then each part is designed in detail.

In short, the design model is a layered blueprint that guides the programmers step by step.

13 **Why is the design model created in levels?**

IMPORTANT

The design model is created in levels because each level hides detail and shows one clear view.

Reviewers can focus on one view at a time instead of being confused by everything at once.

The high-level architecture is agreed first, before time is spent on small details.

This saves effort, because a mistake at a high level would waste all the detailed work below it.

Changes at a high level are cheap, while changes in finished code are costly.

Levels also let different people work in parallel: one team on architecture, another on data design.

Each level becomes a document that guides the next level of work.

Example: if the architecture of a bank app is wrong, every module built on it would need rework.

So the team first agrees on the architecture, and only then designs each component in detail.

In short, levels make design easier to review, cheaper to change and safer to build on.

TOPIC 5

5

Software Architecture

Software architecture is the high-level structure of the system.

It defines the major components and how they connect.

Common styles: layered, client-server, microservices, MVC.

Good architecture makes the system scalable and maintainable.

QUESTION BANK

14 What is software architecture? Why is it important?

IMPORTANT

Software architecture is the high-level structure of the system.

It defines the major components of the software and how they connect and communicate.

It is decided early in the project, before detailed design and coding.

Architecture is important because it is very hard and costly to change later.

Every module is built on top of the architecture, so a wrong architecture affects everything.

A good architecture makes the system scalable, so it can grow with more users.

It also makes the system maintainable, so future changes are easy.

It helps the team divide work, since each component can be built by a different group.

Example: a college portal may use client-server architecture, with a web front end and a database back end.

In short, architecture is the skeleton of the software; a strong skeleton supports a strong system.

15 **Explain any three architectural styles with examples.**

IMPORTANT

An architectural style is a standard pattern for organising the parts of a system.

Layered style: the system is split into layers like presentation, business logic and data.

Each layer talks only to the layer below it, which keeps the design clean.

Example: a typical web app has a browser layer, a server logic layer and a database layer.

Client-server style: clients send requests and the server processes them and replies.

The server holds the data and logic, while clients only show the interface.

Example: an email system where the phone app is the client and Gmail servers do the work.

MVC style: the system is split into Model (data), View (screen) and Controller (logic).

The Model stores data, the View shows it, and the Controller handles user actions.

Example: many web frameworks build apps with a separate model, view and controller.

Each style suits different problems, and the architect picks the one that fits best.

TOPIC 6

6

Data Design

Data design decides how data is organised and stored.

It defines data structures, database tables and relationships.

Good data design makes the software fast and reliable.

It happens before detailed component design.

QUESTION BANK

16 **What is data design?**

IMPORTANT

Data design is the part of design that decides how data is organised and stored.

It is done early, before the detailed design of components.

It defines the data structures the program will use, such as lists, trees and objects.

For database systems, it defines the tables, their fields and the relationships between them.

It also decides which data is stored permanently and which is temporary.

Good data design makes the software fast, because data is easy to find.

It also makes the software reliable, because data stays consistent and safe.

Programmers use the data design as a guide when they write code that handles data.

Example: a library system needs tables for books, members and loans, linked by book id and member id.

In short, data design is the plan for what data exists and where it lives.

17 Give two qualities of a good data design.

IMPORTANT

A good data design has two main qualities: no duplication and fast access.

No duplication means the same fact is stored in only one place.

This avoids inconsistency, where two copies of the same data disagree with each other.

It also saves storage space and makes updates easy, since only one copy needs changing.

Example: a customer address is stored once in the customer table, not copied into every order.

Fast access means frequently used data is organised so it can be found quickly.

This is done using indexes, proper keys and well-planned table structures.

Fast access keeps the software responsive even when the data grows large.

Example: indexing the roll number field lets the system find a student record instantly.

In short, good data design stores each fact once and finds it fast.

TOPIC 7

Architectural Design

7

Architectural design defines the big parts of the system and their connections.

It comes after requirements and before detailed design.

It answers: what are the main components and how do they talk?

The output is often shown with block diagrams.

18 What is architectural design? How is it different from detailed design?

IMPORTANT

Architectural design defines the big parts of the system and how they connect.

It answers: what are the main components, and how do they talk to each other?

It focuses on structure, not on the internal code logic of any part.

Its output is usually shown with block diagrams of components and their links.

Detailed design, on the other hand, describes the inside of each component.

It defines the algorithms, data structures and step-by-step logic of every module.

Architectural design comes first; detailed design comes after it.

In short, architecture decides the shape of the building, while detailed design decides the wiring of each room.

Example: an online shop is first split into user interface, order processing and payment components.

Then detailed design describes exactly how the payment component verifies a card.

Both are needed: architecture gives direction, detailed design gives the code-level plan.

19 Why is architectural design hard to change later?

IMPORTANT

Architectural design is hard to change later because everything is built on top of it.

Every component depends on the architecture for how it connects to other components.

Changing the architecture means changing the interfaces between many components.

That forces rewriting, retesting and re-integrating large parts of the system.

The later the change, the more work is already done, and the more expensive the rework.

Changing architecture late can also break things that were working fine.

This is why architecture is reviewed very carefully before construction starts.

Example: switching a system from one central server to microservices after coding means rebuilding almost everything.

In short, the foundation of a building cannot be changed after the floors are built.

So architects take extra care to get the architecture right the first time.

TOPIC 8

8

UML Class Diagram

A class diagram shows the classes in the system and their relationships.

Each class box has three parts: name, attributes, methods.

Relationships: association (uses), inheritance (is-a), aggregation (has-a), dependency.

It is the most used UML diagram for object-oriented design.

QUESTION BANK

20 **Explain the class diagram with its notation.**

IMPORTANT

A class diagram shows the classes in the system and the relationships between them.

It is the most used UML diagram for object-oriented design.

Each class is drawn as a box divided into three parts.

The top part holds the class name, written in the centre.

The middle part lists the attributes, which are the data the class stores.

The bottom part lists the methods, which are the actions the class can do.

Attributes and methods can show visibility, like + for public and - for private.

Lines between boxes show relationships such as association, inheritance and aggregation.

Example: a Student class box has name and rollNo as attributes and register() as a method.

The diagram gives programmers a clear map of all classes before coding starts.

In short, a class diagram is the blueprint of the object-oriented structure of the software.

21 **Explain the relationships in a class diagram with symbols.**

IMPORTANT

Association means one class uses another class; it is shown by a plain straight line.

The line may have numbers at the ends showing how many objects are linked.

Example: a Student is associated with a Course, since a student enrolls in courses.

Inheritance means a child class extends a parent class, an is-a relationship.

It is shown by a solid line with a hollow arrow pointing to the parent class.

Example: Car inherits from Vehicle, since a car is a vehicle.

Aggregation means a whole-part relationship, a has-a relationship.

It is shown by a solid line with a hollow diamond at the whole end.

Example: a Library has Books; the books can exist even without that library.

Dependency means one class briefly needs another, shown by a dashed arrow.

Example: a Report class depends on a Printer class only while printing.

Knowing these symbols lets a designer read any class diagram correctly.

22 **Draw and explain a class diagram for a library Book class.**

IMPORTANT

To draw the Book class, draw a rectangle box divided into three horizontal parts.

Top part: write the class name, Book, in the centre.

Middle part: list the attributes, which are the data of a book.

Attributes: title, author, price and isIssued, each with its data type.

Bottom part: list the methods, which are the actions a book object can do.

Methods: issue(), returnBook() and displayDetails().

Visibility can be shown: - for private attributes and + for public methods.

If the library has many books, draw an association line from Library to Book with 1 to many multiplicity.

The diagram tells the programmer exactly what data and actions the Book class must have.

Example code-level reading: book.issue() sets isIssued to true when a member borrows it.

In short, this one box is the complete design of the Book class.

TOPIC 9

9

UML Sequence Diagram

A sequence diagram shows how objects interact over time.

Time runs top to bottom; objects sit at the top with lifelines going down.

Messages are arrows between lifelines in time order.

It is perfect for showing one scenario, like a login flow.

23 Explain the sequence diagram with its notation.

IMPORTANT

A sequence diagram shows how objects interact with each other over time.

It is perfect for showing one scenario, like a login flow, step by step.

Time runs from top to bottom in the diagram.

Objects are drawn as boxes at the top of the diagram.

From each object, a dashed vertical line called a lifeline runs downward.

Messages between objects are drawn as horizontal arrows from one lifeline to another.

The arrows are labelled with the message name and are placed in time order.

A reply message is usually shown with a dashed arrow going back.

A tall thin rectangle on a lifeline shows when an object is active.

Example: the User sends login() to the LoginPage, which sends verify() to the Database, which returns the result.

In short, a sequence diagram is a timeline of who talks to whom and in what order.

24 Draw a sequence diagram for a login process.

IMPORTANT

To draw the login sequence, place three boxes at the top: User, LoginPage and Database.

Draw a dashed lifeline going down from each box.

Arrow 1: from User to LoginPage, labelled login(username, password).

Arrow 2: from LoginPage to Database, labelled verify(username, password).

Arrow 3: a dashed reply arrow from Database back to LoginPage, labelled result (valid or invalid).

Arrow 4: from LoginPage back to User, labelled login success or login failed.

If the login fails, an alternative block can show a retry message.

The arrows are read top to bottom, which is the exact order of events.

This diagram shows the programmer the exact steps to code for the login process.

In short, it is a clear visual script of what happens during login.

25 **Distinguish between sequence and collaboration diagrams.**

IMPORTANT

Both diagrams show the same object interactions, but they organise the information differently.

A sequence diagram shows the order of messages over time.

Time flows top to bottom, so the sequence of events is very clear.

It is best when the order of steps matters, like in a login flow.

A collaboration diagram shows how the objects are linked to each other.

Objects are placed freely and connected by lines showing their links.

Messages are written on the links with numbers (1, 2, 3) to show the order.

It is best when the structure of connections matters more than the timeline.

In short, sequence focuses on time order, while collaboration focuses on object links.

A designer can convert one into the other since they carry the same information.

TOPIC 10

10

UML Use Case Diagram

A use case diagram shows what the system offers to its users.

Actors are stick figures outside the system; use cases are ovals inside.

It captures functional requirements from the user point of view.

Relationships: include (must use), extend (optional use).

QUESTION BANK

26 **Explain the use case diagram with its notation.**

IMPORTANT

A use case diagram shows what the system offers to its users.

It captures the functional requirements from the user's point of view.

Actors are drawn as stick figures outside the system boundary.

An actor is anyone or anything that interacts with the system, like a customer or another system.

Use cases are drawn as ovals inside the system boundary box.

Each oval is one goal the actor wants to achieve, like Place Order.

Lines connect each actor to the use cases they take part in.

The include relationship shows a use case that must always be used, drawn with a dashed arrow.

The extend relationship shows an optional use case, also drawn with a dashed arrow.

Example: in a bank system, Customer is an actor and Withdraw Cash is a use case.

In short, the diagram is a simple picture of who can do what with the system.

27 Draw a use case diagram for an online shopping system.

IMPORTANT

Draw a system boundary box and label it Online Shop.

Place two actors outside the box: Customer on the left and Admin on the right.

Inside the box, draw four ovals: Browse Products, Place Order, Make Payment and Manage Stock.

Connect the Customer actor with lines to Browse Products, Place Order and Make Payment.

Connect the Admin actor with a line to Manage Stock.

Add a use case Verify Card inside the box, connected from Make Payment with an include arrow.

This shows that every payment must include card verification.

The diagram clearly shows that customers shop and pay, while the admin manages stock.

It helps the team agree on the system's features before detailed design.

In short, this one picture captures the main functions of the online shop.

28 What is an actor in a use case diagram? Give examples.

IMPORTANT

An actor is a role played by a user or another system that interacts with the system.

It is drawn as a stick figure outside the system boundary.

An actor is not a specific person; it is a role.

The same person can play different actors at different times.

Actors can be primary, like a customer who uses the system directly.

Actors can also be secondary, like a bank server that the system talks to.

Each actor is connected to the use cases it takes part in.

Example: in a college system, Student and Teacher are actors.

A teacher can also be an Examiner actor when setting question papers.

In short, actors define who is outside the system and what they expect from it.

29 What are use case scenarios?

IMPORTANT

A use case scenario is a specific story of how a user interacts with the system to achieve a goal.

The use case names the goal, like withdraw cash; scenarios describe particular paths through it.

The main success scenario describes the happy path where everything works smoothly.

Alternate scenarios describe variations, like choosing a different account.

Exception scenarios describe what happens when things go wrong, like insufficient balance.

Scenarios are written in simple steps: the actor does this, the system responds with that.

They help designers cover every path and help testers derive test cases.

Example: for book a bus ticket, the main scenario is selecting a seat and paying; an exception scenario is payment failure with a retry option.

In short, scenarios turn an abstract use case into concrete stories the team can design and test against.

TOPIC 11

UML Collaboration and Component Diagrams

11

A collaboration diagram shows object interactions with numbered messages on links.

A component diagram shows the physical parts like files and libraries and their dependencies.

Collaboration focuses on structure; sequence focuses on time order.

Component diagrams help plan building and deployment.

QUESTION BANK

30 Explain the collaboration diagram.

IMPORTANT

A collaboration diagram shows how objects interact, focusing on their links.

Objects are drawn as boxes placed freely on the diagram.

Lines called links connect the objects that communicate with each other.

Messages are written on the links with numbers like 1, 2 and 3 to show the order.

It shows the same information as a sequence diagram but arranged by structure, not by time.

It is useful when the designer wants to see which objects are connected.

Example: for login, User, LoginPage and Database boxes are linked, with 1: login(), 2: verify(), 3: result.

The numbered messages let the reader follow the order of events.

In short, a collaboration diagram is a map of object links with ordered messages on them.

Designers often draw it when the connection structure matters most.

31 Explain the component diagram with an example.

IMPORTANT

A component diagram shows the physical parts of the system and their dependencies.

Physical parts include executables, libraries, files and database drivers.

Each component is drawn as a box, sometimes with a small component symbol.

Dependency arrows between boxes show which part needs which other part.

It helps plan how the system will be built, assembled and deployed.

It shows the impact of change: if one component changes, its dependents are affected.

Example: a billing app component depends on the payment library and the database driver.

If the payment library is updated, the diagram shows the billing app must be retested.

It is drawn late in design, when the physical structure of the code is known.

In short, a component diagram is a map of the buildable pieces and how they fit together.

TOPIC 12

Testing Strategy

12

Testing strategy is the overall plan for finding defects.

It starts with unit testing, then integration, then validation, then system testing.

The strategy answers: what to test, when, and with what resources.

A good strategy finds bugs early when they are cheap to fix.

QUESTION BANK

32 Explain unit, integration, validation and system testing. [Asked: May 2019, Oct 2021]

REPEATED

Unit testing tests each small part of the software in isolation.

It is usually done by the developer who wrote the code.

It catches bugs early, when they are cheapest to fix.

Example: testing a single calculateTotal() function with different inputs.

Integration testing tests combined modules working together.

It checks that data passes correctly between modules.

Example: testing that the order module correctly calls the payment module.

Validation testing checks that the software meets the customer's requirements.

It answers: did we build the right product?

System testing tests the complete integrated system as one whole.

It is done in an environment similar to the real user setup.

Example: testing the full online shop from browsing to delivery on real servers.

In short, the levels move from small pieces to the whole system, catching different bugs at each step.

33 Explain verification and validation in detail. [Asked: Mar 2021, Aug 2022]

REPEATED

Verification and validation are the two complementary checks that software is built correctly.

Verification asks: are we building the product right? It checks work products against their specifications.

It includes reviews, inspections and walkthroughs, all done without running the code.

Validation asks: are we building the right product? It checks the software against user needs.

It is done by actually executing the software through testing.

Verification catches errors early in documents and design, when fixing them is cheap.

Validation catches errors in behaviour, when the software meets real users and real data.

Both are needed because a product can meet its specification yet still disappoint the user.

Example: verifying a design document for missing screens is verification; running the app to see if booking works is validation.

In short, verification checks the papers, validation checks the product, and both are needed for quality.

34 What are the objectives of software testing? [Asked: Model paper]

REPEATED

The main objective of testing is to find errors in the software before delivery.

It verifies that the software meets the requirements agreed with the customer.

It validates that the software behaves correctly under real conditions.

It builds confidence that the product is fit for use.

It measures quality by counting and classifying the defects found.

It prevents future failures by catching problems when they are still cheap to fix.

It also checks non-functional needs like speed, security and usability.

Example: the objective of testing a payment app is not to prove it works, but to hunt down every way money could go missing.

In short, testing aims to expose weaknesses early so the delivered software can be trusted.

35 What is a testing strategy? Explain the levels of testing.

IMPORTANT

A testing strategy is the overall plan that organises all testing work.

It answers: what to test, when to test, who tests it and what counts as pass.

Testing is done in levels, from small parts up to the full system.

Unit testing: each small module is tested alone by the developer.

Integration testing: combined modules are tested to check they work together.

Validation testing: the software is checked against the requirements.

System testing: the full system is tested end to end in a real-like environment.

A good strategy finds bugs early, when they are cheapest to fix.

It also defines the test environment, tools and resources needed.

Example: a bank app plan would test each function first, then the full transfer flow.

In short, the strategy is the roadmap that guides testing from code to delivery.

36 Is complete testing of a software possible? Explain why.

IMPORTANT

Complete testing of software is not possible in practice.

The number of possible inputs is usually infinite: every number, text and click combination cannot be tried.

All execution paths cannot be covered because loops and combinations multiply beyond any test budget.

Time and money are limited, so testing must stop at a reasonable point.

Instead of completeness, testers aim for risk-based coverage of the most important paths and inputs.

Techniques like equivalence partitioning and boundary value analysis choose a smart sample instead of everything.

The goal shifts from proving no bugs exist to finding the important bugs early.

Example: a simple calculator with two number fields has billions of input pairs; no team can test them all.

In short, exhaustive testing is a theoretical idea; real testing samples the most valuable cases within the available time.

37 What is software testing?

IMPORTANT

Software testing is the process of running a program to find errors before users do.

Its goal is to uncover defects in requirements, design and code.

Testing checks both that the software does what it should and that it does not do what it should not.

It is done at several levels: unit, integration, validation and system testing.

Both manual testing by people and automated testing by scripts are used.

Good testing needs planned test cases with expected results, not random clicking.

Testing also measures quality by recording and classifying the defects found.

Example: testing a login page means trying correct passwords, wrong passwords, empty fields and locked accounts.

In short, testing is the safety net that catches mistakes before the customer pays for them.

38 Which types of errors are detected in software testing?

IMPORTANT

Testing detects several types of errors at different levels of the software.

Requirements errors: the software was built from misunderstood or missing requirements.

Design errors: the structure is wrong, like a missing module or a bad interface.

Coding errors: bugs in program logic, wrong calculations or mishandled conditions.

Interface errors: modules fail to communicate correctly with each other.

Performance errors: the software is too slow or uses too much memory.

Documentation errors: the manual or help text does not match what the software does.

Security errors: the software allows unauthorised access or leaks data.

Example: a fee portal might calculate totals wrong, which is a coding error, or show a confusing receipt, which is an interface error.

In short, testing hunts errors all the way from misunderstood requirements down to wrong lines of code.

TOPIC 13

Test Strategies for Conventional Software

13

Conventional software follows the classic test levels in order.

Top-down integration: test upper modules first using stubs for lower ones.

Bottom-up integration: test lower modules first using drivers for upper ones.

Big-bang integration tests everything together at once (risky).

QUESTION BANK

39 Explain test strategies for conventional software.

IMPORTANT

Test strategies for conventional software follow the levels in order: unit, integration, then system.

Unit testing is done first on each module separately.

Integration testing then combines the modules step by step.

Top-down integration starts from the main module and works downward.

Missing lower modules are replaced with stubs, which are dummy modules.

Bottom-up integration starts from the lowest modules and works upward.

Missing upper modules are replaced with drivers, which are dummy calling programs.

Big-bang integration combines everything at once and tests it together.

Big-bang is risky because when a failure happens, it is hard to find which module caused it.

Example: testing a main menu with stubs for payment and stock is top-down testing.

In short, gradual integration finds bugs early, while big-bang finds them late and painfully.

40 What are stubs and drivers in integration testing?

IMPORTANT

A stub is a dummy lower-level module used in top-down integration testing.

It pretends to be the real module and returns simple fixed answers.

This lets the upper modules be tested before the lower ones are coded.

A driver is a dummy calling program used in bottom-up integration testing.

It calls the lower module and passes test data to it.

This lets the lower modules be tested before the upper ones are ready.

Example: testing a main menu with a stub that pretends the payment module always says payment ok.

Example: testing a payment function with a driver program that calls it with test card numbers.

Stubs and drivers are thrown away once the real modules are ready.

In short, stubs stand in for modules below, and drivers stand in for modules above.

TOPIC 14

Black Box vs White Box Testing

14

Black box testing checks behaviour without seeing the code.

White box testing checks the internal code logic and paths.

Black box is done from the user view; white box needs code knowledge.

Both are needed: neither alone finds all bugs.

QUESTION BANK

41 **Explain black box testing techniques with examples. [Asked: May 2019, Oct 2021, Aug 2022]**

SURE MARK

Black box testing uses techniques based on inputs and outputs, not code.

Equivalence partitioning divides inputs into groups that should behave the same.

The tester picks one value from each group, which reduces the number of tests.

Example: for an age field 18 to 60, groups are below 18, 18 to 60, and above 60.

Testing 15, 30 and 70 covers all three groups.

Boundary value analysis tests the values at the edges of each group.

Most bugs hide at boundaries, so these values are tested carefully.

Example: for a 1 to 100 field, test 0, 1, 2, 99, 100 and 101.

Another example: for age 18 to 60, test 17, 18, 60 and 61.

These techniques find input-handling bugs without ever reading the code.

In short, test one from each group and always test the edges.

42 **Explain white box testing techniques with examples. [Asked: May 2019, Mar 2021, Oct 2021, Aug 2022, Feb 2025]**

SURE MARK

White box testing uses techniques based on the internal code structure.

Statement coverage requires every line of code to run at least once.

It is the weakest measure; running a line does not mean it is correct.

Branch coverage requires every decision outcome to be tested, both true and false.

Example: an if-else needs two tests, one for each branch.

Path coverage requires every possible route through the code to be tested.

It is the strongest but also the hardest, since paths can be very many.

Condition coverage tests each individual condition inside a decision.

Example: for if (a > 0 and b > 0), test all combinations of a and b.

These techniques need the tester to read and understand the code.

In short, white box testing makes sure no part of the code is left untested.

43 Distinguish between black box and white box testing.

IMPORTANT

Black box testing checks the software's behaviour without looking at the code.

The tester knows only the inputs and the expected outputs.

It is done from the user's point of view.

It finds missing functions, wrong outputs and interface errors.

Example: testing login by trying correct and wrong passwords.

White box testing checks the internal logic and structure of the code.

The tester sees the code and designs tests for its paths and branches.

It is done from the developer's point of view and needs code knowledge.

It finds hidden logic errors, untested paths and dead code.

Example: testing every if-else branch inside the login function.

In short, black box asks what the software does, while white box asks how it does it.

Both are needed because neither one alone finds all types of bugs.

44 Why are both black box and white box testing needed?

IMPORTANT

Black box and white box testing find different types of bugs, so both are needed.

Black box testing finds missing functions and wrong behaviour.

It checks the software from the user's view, so it catches usability and requirement errors.

Example: black box testing reveals that the forgot password feature is missing.

White box testing finds hidden logic errors inside the code.

It checks paths and branches that black box tests may never reach.

Example: white box testing reveals a division by zero in a rarely used branch.

Black box alone cannot find code-level bugs; white box alone cannot find missing features.

Using both gives the best defect detection and higher confidence.

In short, black box checks what the software does, white box checks how, and together they cover everything.

45 What is meant by loop testing? Explain.

IMPORTANT

Loop testing is a white-box technique that focuses on the validity of loop constructs in code.

Simple loops are tested with five cases: skip the loop entirely, one pass, two passes, a typical number of passes, and the maximum passes.

Nested loops are tested from the innermost loop outward, keeping outer loops at minimum values.

Concatenated loops are tested as separate simple loops when they are independent.

Unstructured loops are redesigned into structured form before testing wherever possible.

The aim is to catch off-by-one errors and wrong termination conditions.

Example: a loop that processes 100 student records is tested with 0, 1, 2, 50 and 100 records.

In short, loop testing systematically exercises loop boundaries, where many defects hide.

46 Explain flow graph with an example.

IMPORTANT

A flow graph is a diagram that shows the logical flow of a program for white-box testing.

Each circle, called a node, represents one or more program statements.

Arrows, called edges, show the flow of control from one node to another.

Areas bounded by edges and nodes are called regions, including the outside area.

The cyclomatic complexity is computed as edges minus nodes plus 2, giving the number of independent paths.

Each independent path becomes a test case in path testing.

Example: for a simple if-else that checks pass or fail marks, the flow graph has 4 nodes and the complexity is 2, so two test cases cover both paths.

In short, a flow graph turns code structure into a map that tells testers exactly which paths to test.

TOPIC 15

Validation and System Testing

15

Validation testing checks the software meets the requirements.

Alpha testing: done by developers at their site with users.

Beta testing: done by real users at their own place.

System testing tests the complete integrated system end to end.

QUESTION BANK

47 **What is system testing? What does it include? [Asked: Model paper]**

SURE MARK

System testing checks the fully integrated system as one complete whole.

It is done after integration testing, on the entire software plus hardware.

It is performed in an environment similar to the real user setup.

It includes performance testing, which checks speed under normal and heavy load.

It includes security testing, which checks that data is protected from attacks.

It includes recovery testing, which checks the system restarts correctly after a crash.

It also includes stress testing, usability testing and compatibility testing.

The goal is to prove the whole system works in real-world conditions.

Example: testing a railway booking site with thousands of users booking at once.

In short, system testing is the final check that the complete system is ready for the customer.

48 **Explain validation testing with alpha and beta testing. [Asked: May 2019, Aug 2022]**

REPEATED

Validation testing checks that the software meets the customer's requirements.

It answers: did we build the right product?

It is done after integration testing, on the nearly finished software.

Alpha testing is done by developers at the developer's site with users present.

Users try the software while the team watches and notes problems.

It happens before the software is released to the public.

Beta testing is done by real users at their own place.

The software is given to selected users who report bugs from daily use.

Example: a new app given to 100 students before launch is beta testing.

Alpha finds bugs in a controlled setting; beta finds bugs in real-world use.

In short, validation with alpha and beta testing proves the software is ready for release.

49 Explain functional testing and acceptance testing. [Asked: Mar 2021, Feb 2025]

REPEATED

Functional testing checks what the software does; acceptance testing checks whether the customer accepts it.

Functional testing verifies each function against its specification, using black-box techniques like equivalence partitioning.

It is usually done by the test team during system testing.

Acceptance testing is done by the customer or end users before final delivery.

Alpha testing is acceptance testing done at the developer's site with the customer present.

Beta testing is acceptance testing done at customer sites under real conditions.

Functional testing can pass while acceptance testing fails, if the software is correct but unusable.

Example: functional testing checks that a 10 percent discount applies correctly; acceptance testing is the shop owner confirming the billing screen is usable for daily work.

In short, functional testing proves the features work, acceptance testing proves the customer is satisfied.

50 What is recovery testing? Explain.

IMPORTANT

Recovery testing checks whether software can recover correctly after a failure.

The system is forced to fail in various ways: power cut, network drop or disk full.

Testers then check that it restarts and continues without data loss or corruption.

It verifies that recovery procedures like backups, checkpoints and transaction rollbacks actually work.

It also checks that the system returns to a known good state within an acceptable time.

It is a type of system testing focused on reliability rather than features.

Example: a banking app is tested by cutting the network during a transfer; it must roll back cleanly so no money is lost or duplicated.

In short, recovery testing proves the software can fall down safely and get back up.

51 What is robustness testing? Explain.

IMPORTANT

Robustness testing checks how software behaves with invalid or unexpected inputs.

It feeds wrong data types, extreme values and empty inputs to see whether the program crashes.

The software should handle bad input gracefully with clear error messages, not freeze or corrupt data.

It is a black-box technique because it needs no knowledge of the code inside.

It protects against both mistaken users and deliberate misuse.

Test cases include boundary breakers like huge numbers, special characters and null values.

Example: entering letters in a phone-number field or a birth year of 3000 should show a polite error, not crash the app.

In short, robustness testing proves the software stays standing when users do unexpected things.

TOPIC 16

Debugging

16

Debugging is finding and fixing the cause of a test failure.

Steps: reproduce the bug, find the cause, fix it, retest.

Techniques: print statements, breakpoints, going backwards from the failure.

Testing finds the bug; debugging removes it.

QUESTION BANK

52 What is debugging? Explain the debugging process.

IMPORTANT

Debugging is the process of finding the cause of an error and fixing it.

It starts after testing reports a failure.

Step 1: reproduce the bug, so it can be seen happening again.

Step 2: locate the cause by studying the code around the failure.

Step 3: fix the faulty code with the smallest safe change.

Step 4: retest to confirm the bug is gone and nothing else broke.

Common techniques include adding print statements to trace values.

Breakpoints let the developer pause the program and inspect variables.

Backtracking means going backwards from the failure to find its origin.

Example: a wrong total in a bill is traced back to a faulty discount calculation.

In short, debugging is detective work: find the cause, fix it, and prove it is fixed.

53 Distinguish between testing and debugging.

IMPORTANT

Testing and debugging are related but different activities.

Testing is the process of running the software to find bugs.

It shows that a bug exists by running test cases with known inputs.

Testing can be done by testers as well as developers.

Debugging is the process of finding where the bug is and fixing it.

It starts after testing reports a failure.

Debugging is done by developers, since it needs code knowledge.

In short, testing finds the symptom, while debugging finds and cures the disease.

Example: testing shows the login fails; debugging finds the wrong password check in the code.

A good team does both well: testing to catch bugs, debugging to remove them.

TOPIC 17

Software Quality and Product Metrics

17

Software quality means the software does what users need, reliably.

Product metrics measure the software itself, not the process.

Common metrics: defects found, defect density, test coverage.

Metrics help managers decide if the software is ready to release.

QUESTION BANK

54 What is the significance of design model metrics? [Asked: Model paper]

REPEATED

Design model metrics measure the quality of the software design before any code is written.

They help catch design flaws early, when fixing them is still cheap.

They measure complexity, so overly complicated designs can be simplified in time.

They measure coupling and cohesion, guiding the designer toward independent, focused modules.

They give an objective score instead of relying only on personal judgement.

They help estimate testing effort, since complex designs need more test cases.

They allow comparison between alternative designs on facts rather than opinions.

Example: if the metrics show one module connected to twenty others, the designer splits it before coding starts.

In short, design model metrics are an early warning system for bad designs.

55 What is software quality? Explain its factors.

IMPORTANT

Software quality means the software meets user needs correctly and reliably.

Correctness: the software does exactly what the requirements ask.

Reliability: it works without failing for a long time.

Usability: it is easy to learn and easy to use.

Efficiency: it uses time and memory wisely without being slow.

Maintainability: it is easy to fix and change in the future.

Portability: it can run on different machines and platforms.

High quality software has few defects and satisfies its users.

Low quality software is full of bugs and frustrates users.

Example: a banking app must be correct and reliable above all else.

In short, quality is not just working software, but software that works well for its users.

56 What are product metrics? Give examples.

IMPORTANT

Product metrics are measurements of the software product itself, not the process.

They help managers judge the quality of the software.

Common examples: the total number of defects found during testing.

Defect density: defects per thousand lines of code, which allows fair comparison.

Example: 5 defects in 1000 lines means a defect density of 5.

Test coverage: the percentage of code or requirements covered by tests.

Higher coverage usually means fewer hidden bugs.

They also include measures like reliability and response time.

Metrics tell whether quality is improving or getting worse over time.

They help decide if the software is ready to release.

In short, product metrics turn quality into numbers that can be tracked.

TOPIC 18

Metrics Across the Life Cycle

18

Analysis model metrics: check the requirements model, like number of use cases.

Design model metrics: check design quality, like coupling between modules.

Source code metrics: like lines of code and complexity per function.

Testing and maintenance metrics: like defects found per test hour.

QUESTION BANK

57 Explain metrics for analysis model, design model, source code, testing and maintenance.

IMPORTANT

Different metrics are used at each stage of the software life cycle.

Analysis model metrics check the requirements model.

They measure size and completeness, like the number of use cases and requirements.

Design model metrics check the quality of the design.

They measure modularity, like coupling between modules and cohesion inside modules.

Source code metrics measure the code itself.

They include lines of code and the complexity of each function.

Testing metrics measure the testing effort and results.

They include defects found, test coverage and defects per test hour.

Maintenance metrics measure how easy the software is to fix.

They include the time taken to fix a defect and the number of change requests.

In short, each phase has its own metrics, and together they track quality from start to finish.

58 Write a short note on code metrics.

IMPORTANT

Code metrics are measurements taken from the source code.

Lines of code is the simplest metric: it counts the size of the program.

It is easy to measure but a long program is not always worse than a short one.

Complexity metrics count the decision points in the code, like ifs and loops.

Higher complexity means the code is harder to understand and test.

Example: a function with 10 if statements is more complex than one with 2.

Defect density measures defects per 1000 lines of code.

It allows fair quality comparison between projects of different sizes.

Example: project A with density 2 is better than project B with density 8.

Code metrics help managers spot risky modules that need more testing.

In short, code metrics turn the code into numbers that guide quality decisions.